

3D Rotations and Attitude Representation

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 22, 2026

Algorithm Recap

We have seen two main trajectory optimization approaches:

- Indirect / Local methods (DDP, iLQR)
- Direct methods (DIRCOL + SQP)

Comparison:

- DDP: dynamically feasible at every iteration
- Direct Collocation: feasibility enforced via constraints
- SQP: general constrained Newton method

Now we discuss 3D rotation issues.

What is Attitude?

Attitude = rotation from body frame to world frame.

$$x_{world} = R x_{body}$$

Rotation has:

3 DOF

But:

There is no globally nonsingular 3-parameter representation.

Rotation Matrix: $SO(3)$

Rotation matrix:

$$R \in \mathbb{R}^{3 \times 3}$$

Orthogonality:

$$R^T R = I$$

Determinant:

$$\det(R) = 1$$

Set:

$$SO(3) = \{R \mid R^T R = I, \det R = 1\}$$

3 DOF, but 9 variables $\rightarrow$ constraints.

Angular Velocity and Skew Operator

Define angular velocity:

$$\omega = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

Skew-symmetric matrix:

$$[\omega]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

Property:

$$[\omega]_{\times} v = \omega \times v$$

Rotation Matrix Kinematics

Body angular velocity:

$$\dot{R} = R[\omega]_{\times}$$

Derivation:

For body vector v_b :

$$v_w = Rv_b$$

Differentiate:

$$\dot{v}_w = \dot{R}v_b$$

But also:

$$\dot{v}_w = \omega \times v_w$$

$$= [\omega]_{\times} Rv_b$$

Thus:

Axis-Angle Representation

Define:

$$\text{Axis } a \in \mathbb{R}^3, \quad \|a\| = 1$$

$$\text{Angle } \theta$$

Rotation vector:

$$\phi = \theta a$$

Rodrigues formula:

$$R = I + \sin \theta [a]_{\times} + (1 - \cos \theta) [a]_{\times}^2$$

Quaternions

Define quaternion:

$$q = \begin{bmatrix} q_0 \\ \mathbf{q} \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ a \sin(\theta/2) \end{bmatrix}$$

Constraint:

$$\|q\| = 1$$

Note:

$$q \equiv -q$$

(Double cover of $SO(3)$)

Quaternion Multiplication

For:

$$q = \begin{bmatrix} q_0 \\ \mathbf{q} \end{bmatrix}, \quad p = \begin{bmatrix} p_0 \\ \mathbf{p} \end{bmatrix}$$

Product:

$$q \otimes p = \begin{bmatrix} q_0 p_0 - \mathbf{q}^T \mathbf{p} \\ q_0 \mathbf{p} + p_0 \mathbf{q} + \mathbf{q} \times \mathbf{p} \end{bmatrix}$$

Quaternion Conjugate

$$q^* = \begin{bmatrix} q_0 \\ -\mathbf{q} \end{bmatrix}$$

Inverse (unit quaternion):

$$q^{-1} = q^*$$

Rotating a Vector

Represent vector as quaternion:

$$v_q = \begin{bmatrix} 0 \\ v \end{bmatrix}$$

Rotation:

$$v_{rot} = q \otimes v_q \otimes q^*$$

Equivalent to:

$$v_{rot} = R(q)v$$

Quaternion Kinematics

Angular velocity quaternion:

$$\omega_q = \begin{bmatrix} 0 \\ \omega \end{bmatrix}$$

Kinematic equation:

$$\dot{q} = \frac{1}{2} q \otimes \omega_q$$

Expanded:

$$\dot{q}_0 = -\frac{1}{2} \mathbf{q}^T \omega$$

$$\dot{\mathbf{q}} = \frac{1}{2} (q_0 \omega + \mathbf{q} \times \omega)$$

Why Quaternions in Optimization?

Advantages:

- No singularities
- Compact (4 parameters)
- Easy normalization
- Good numerical stability

Used in:

- DDP
- Direct Collocation
- MPC for aerial robots